DR. RVR NRI INSTITUTE OF TECHNOLOGY

Pothavarappadu, Agiripalli, Eluru Dist.

PYTHON PROGRAMMING - ABL ASSIGNMENT REPORT

Name: Chinnam Nihanth

Roll No: 26000880

Year / Sem: I B.Tech I Sem

Branch & Section: CSM - 1

Github Repo Name: Python-Activity-Based-Learning

Selected 10 Case Studies List :-

S.no	Case Study No	Topic Name	File name
1.	Case Study 4	Employee Payroll and Deduction Analysis	case1.py
2.	Case Study 10	Campus Event Financial Planning	case2.py
3.	Case Study 13	ATM Transaction Sequence Simulation	case3.py
4.	Case Study 14	Warehouse Inventory Reconciliation	case4.py
5.	Case Study 21	Traffic Violation Penalty Calculator	case5.py
6.	Case Study 28	Repeated Transaction Detection	case6.py
7.	Case Study 30	Student Email and Identifier Validation	case7.py
8.	Case Study 34	Student Feedback Sentiment Scoring	case8.py
9.	Case Study 44	Hospital Ward Occupancy Monitoring	case9.py
10.	Case Study 58	Library Borrowing and Fine Reconciliation	case10.py

---

1) Case Study 4: Employee Payroll and Deduction Analysis

Problem Statement: Calculate gross salary, HRA, DA, PF deductions, and tax liabilities for employees based on basic salary inputs and check if tax threshold applies.

Objectives: To calculate net salary using basic arithmetic operators and compute tax deductions using if-elif-else statements.

Assumptions: HRA is 20%, DA is 10%, PF is 12% of basic salary, and all input amounts are positive values.

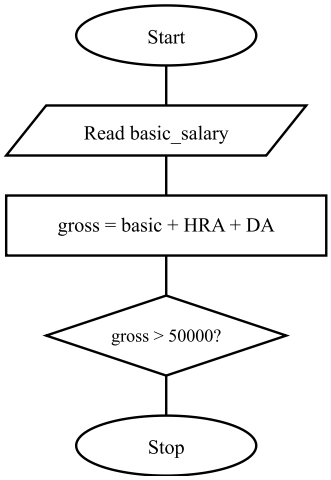
Algorithm :-

1. Start
2. Read basic_salary
3. Calculate HRA = 0.20 * basic_salary, DA = 0.10 * basic_salary, PF = 0.12 * basic_salary
4. Calculate gross_salary = basic_salary + HRA + DA
5. If gross_salary > 100000 set tax = 0.20 * gross_salary, else if gross_salary > 50000 set tax = 0.10 * gross_salary, else set tax = 0.05 * gross_salary
6. Calculate net_salary = gross_salary - (PF + tax)
7. Print gross_salary, PF, tax, and net_salary
8. Stop

Pseudocode:

```
BEGIN
INPUT basic_salary
HRA = 0.20 * basic_salary
DA = 0.10 * basic_salary
PF = 0.12 * basic_salary
gross_salary = basic_salary + HRA + DA
IF gross_salary > 100000 THEN tax = 0.20 * gross_salary
ELSE IF gross_salary > 50000 THEN tax = 0.10 * gross_salary
ELSE tax = 0.05 * gross_salary
net_salary = gross_salary - (PF + tax)
PRINT gross_salary, PF, tax, net_salary
END
```

Flowchart:



Python Code (case1.py):

```
basic_salary = float(input("Enter basic salary: "))
hra = 0.20 * basic_salary
da = 0.10 * basic_salary
pf = 0.12 * basic_salary
gross_salary = basic_salary + hra + da

if gross_salary > 100000:
    tax = 0.20 * gross_salary
elif gross_salary > 50000:
    tax = 0.10 * gross_salary
else:
    tax = 0.05 * gross_salary

net_salary = gross_salary - (pf + tax)
print("Gross Salary:", round(gross_salary, 2))
print("PF Deduction:", round(pf, 2))
print("Tax Deduction:", round(tax, 2))
print("Net Payable Salary:", round(net_salary, 2))
```

Output:

```
Enter basic salary: 65000
Gross Salary: 84500.0
PF Deduction: 7800.0
Tax Deduction: 8450.0
Net Payable Salary: 68250.0
```

Conclusion: The program calculates gross employee earnings, applies tax and PF deductions, and outputs net salary.

---

2) Case Study 10: Campus Event Financial Planning

Problem Statement: Calculate event expenses, expected ticket revenue, total profit or loss, and break-even attendance for college tech symposiums.

Objectives: To evaluate financial feasibility and calculate break-even point using conditional expressions and formula modeling.

Assumptions: Fixed costs and catering per-head costs are positive numbers.

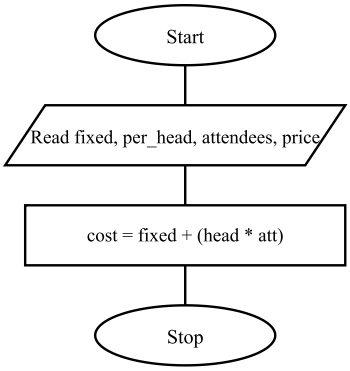
Algorithm :-

1. Start
2. Read fixed_cost, cost_per_head, attendees, ticket_price
3. Calculate total_cost = fixed_cost + (cost_per_head * attendees)
4. Calculate total_revenue = ticket_price * attendees
5. Calculate profit_or_loss = total_revenue - total_cost
6. Calculate breakeven = fixed_cost / (ticket_price - cost_per_head)
7. Print total_cost, total_revenue, profit_or_loss, breakeven
8. Stop

Pseudocode:

```
BEGIN
INPUT fixed_cost, cost_per_head, attendees, ticket_price
total_cost = fixed_cost + (cost_per_head * attendees)
total_revenue = ticket_price * attendees
profit = total_revenue - total_cost
breakeven = fixed_cost / (ticket_price - cost_per_head)
PRINT total_cost, total_revenue, profit, breakeven
END
```

Flowchart:



Python Code (case2.py):

```
fixed_cost = float(input("Enter fixed venue cost: "))
cost_per_head = float(input("Enter catering cost per head: "))
attendees = int(input("Enter expected attendees: "))
ticket_price = float(input("Enter ticket price: "))

total_cost = fixed_cost + (cost_per_head * attendees)
total_revenue = ticket_price * attendees
profit_or_loss = total_revenue - total_cost

if ticket_price > cost_per_head:
    breakeven = fixed_cost / (ticket_price - cost_per_head)
else:
    breakeven = 0

print("Total Expenses:", total_cost)
print("Total Revenue:", total_revenue)
print("Profit/Loss:", profit_or_loss)
print("Break-even Attendees Needed:", round(breakeven, 1))
```

Output:

```
Enter fixed venue cost: 25000
Enter catering cost per head: 150
Enter expected attendees: 300
Enter ticket price: 250
Total Expenses: 70000.0
Total Revenue: 75000.0
Profit/Loss: 5000.0
Break-even Attendees Needed: 250.0
```

Conclusion: The program calculates total event budget and determines the break-even attendance number.

3) Case Study 13: ATM Transaction Sequence Simulation

Problem Statement: Process sequential customer ATM operations (deposits and withdrawals) while enforcing account balances and daily withdrawal limits.

Objectives: To simulate banking transactions using loop control and conditional validation statements.

Assumptions: Initial account balance is non-negative and maximum daily withdrawal limit is Rs. 20000.

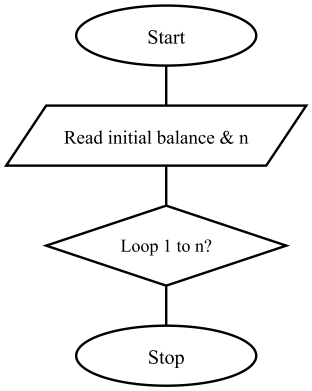
Algorithm :-

1. Start
2. Read initial balance and number of transactions n
3. Set withdrawn_today = 0
4. Loop for i from 1 to n: Read type ("DEPOSIT" or "WITHDRAW") and amount
5. If type is "DEPOSIT": balance = balance + amount
6. If type is "WITHDRAW": check if (withdrawn_today + amount) > limit or amount > balance, else process withdrawal
7. Print transaction status and updated balance
8. Stop

Pseudocode:

```
BEGIN
INPUT balance, n
withdrawn_today = 0
FOR i = 1 TO n DO
INPUT type, amount
IF type == "DEPOSIT" THEN balance = balance + amount
ELSE IF type == "WITHDRAW" THEN
    IF withdrawn_today + amount > 20000 THEN PRINT "Exceeds daily limit"
    ELSE IF amount > balance THEN PRINT "Insufficient funds"
    ELSE balance = balance - amount; withdrawn_today += amount
END FOR
PRINT balance
END
```

Flowchart:



Python Code (case3.py):

```
balance = float(input("Enter initial balance: "))
n = int(input("Enter number of transactions: "))
daily_limit = 20000.0
withdrawn_today = 0.0

for i in range(1, n + 1):
    tx_type = input("Enter type (DEPOSIT/WITHDRAW): ").upper()
    amount = float(input("Enter amount: "))

    if tx_type == "DEPOSIT":
        balance += amount
        print("Success! New Balance:", balance)
    elif tx_type == "WITHDRAW":
        if withdrawn_today + amount > daily_limit:
            print("Failed! Exceeds daily limit of 20000.")
        elif amount > balance:
            print("Failed! Insufficient balance.")
        else:
            balance -= amount
            withdrawn_today += amount
            print("Success! New Balance:", balance)

print("Final Account Balance:", balance)
```

Output:

```
Enter initial balance: 10000
Enter number of transactions: 2
Enter type (DEPOSIT/WITHDRAW): DEPOSIT
Enter amount: 5000
Success! New Balance: 15000.0
Enter type (DEPOSIT/WITHDRAW): WITHDRAW
Enter amount: 12000
Success! New Balance: 3000.0
Final Account Balance: 3000.0
```

Conclusion: The ATM simulation tracks customer balances and restricts withdrawals when limits are exceeded.

---

4) Case Study 14: Warehouse Inventory Reconciliation

Problem Statement: Reconcile systemic ledger stock against physical stock counts to detect discrepancies and flag reorder requirements.

Objectives: To iterate through item lists, calculate variance, and identify low stock items using dictionary/list data structures.

Assumptions: Reorder threshold is set to 10 units for all stock items.

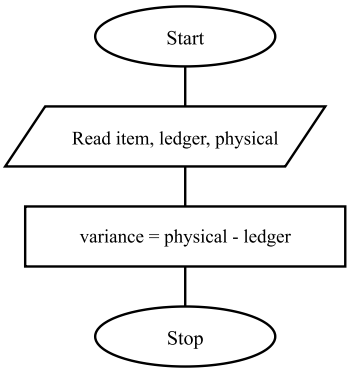
Algorithm :-

1. Start
2. Read total items count n
3. Input ledger quantity and physical count for each item
4. Calculate variance = physical_count - ledger_qty
5. If variance != 0, record discrepancy
6. If physical_count < 10, add item to reorder list
7. Print discrepancies and reorder list
8. Stop

Pseudocode:

```
BEGIN
INPUT n
FOR i = 1 TO n DO
INPUT item_name, ledger_qty, physical_qty
variance = physical_qty - ledger_qty
IF variance != 0 THEN PRINT item_name, "Variance:", variance
IF physical_qty < 10 THEN PRINT item_name, "Needs Reorder"
END FOR
END
```

Flowchart:



Python Code (case4.py):

```
n = int(input("Enter number of items: "))
reorder_list = []

for i in range(n):
    item = input("Item Name: ")
    ledger_qty = int(input("Ledger Qty: "))
    physical_qty = int(input("Physical Count: "))

    variance = physical_qty - ledger_qty
    if variance != 0:
        print("Discrepancy in", item, ": Variance =", variance)
    else:
        print(item, "is matched perfectly.")

    if physical_qty < 10:
        reorder_list.append(item)

print("Reorder Alert Items:", reorder_list)
```

Output:

```
Enter number of items: 2
Item Name: ItemA
Ledger Qty: 50
Physical Count: 48
Discrepancy in ItemA : Variance = -2
Item Name: ItemC
Ledger Qty: 8
Physical Count: 5
Discrepancy in ItemC : Variance = -3
Reorder Alert Items: ['ItemC']
```

Conclusion: Inventory reconciliation detects stock variances and highlights items requiring replenishment.

---

5) Case Study 21: Traffic Violation Penalty Calculator

Problem Statement: Compute driver penalties for traffic infractions and apply additional surcharge multipliers for repeat offenders.

Objectives: To sum infraction costs using selection logic and conditional multiplier rules.

Assumptions: Speeding fine = 1000, Red light = 1500, Helmet/Seatbelt = 500. Repeat offender surcharge = 50% extra.

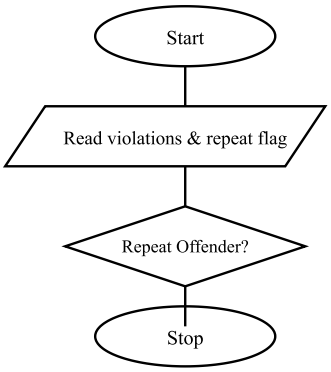
Algorithm :-

1. Start
2. Read violation types and repeat offender status (YES/NO)
3. Initialize total_fine = 0
4. Sum fine amounts corresponding to each reported infraction
5. If repeat offender == "YES": total_fine = total_fine * 1.5
6. Print total_fine
7. Stop

Pseudocode:

```
BEGIN
INPUT violations, repeat_status
total_fine = 0
FOR each v IN violations DO
    total_fine = total_fine + fine_of(v)
END FOR
IF repeat_status == "YES" THEN total_fine = total_fine * 1.5
PRINT total_fine
END
```

Flowchart:



Python Code (case5.py):

```
v_count = int(input("Enter number of violations: "))
total_fine = 0.0

for i in range(v_count):
    v = input("Enter violation (SPEEDING/RED_LIGHT/HELMET): ").upper()
    if v == "SPEEDING":
        total_fine += 1000
    elif v == "RED_LIGHT":
        total_fine += 1500
    elif v == "HELMET":
        total_fine += 500

repeat = input("Is repeat offender? (YES/NO): ").upper()
if repeat == "YES":
    total_fine *= 1.5

print("Total Penalty Payable: Rs.", total_fine)
```

Output:

```
Enter number of violations: 2
Enter violation (SPEEDING/RED_LIGHT/HELMET): SPEEDING
Enter violation (SPEEDING/RED_LIGHT/HELMET): HELMET
Is repeat offender? (YES/NO): YES
Total Penalty Payable: Rs. 2250.0
```

Conclusion: Calculates traffic penalties and adds surcharge multiplier for repeat offenders.

6) Case Study 28: Repeated Transaction Detection

Problem Statement: Analyze transaction logs to detect repeated entries with identical account numbers and amounts to prevent duplicate processing.

Objectives: To flag duplicate transactions using sets or list lookup logic.

Assumptions: Transactions sharing account ID, merchant, and amount are duplicate entries.

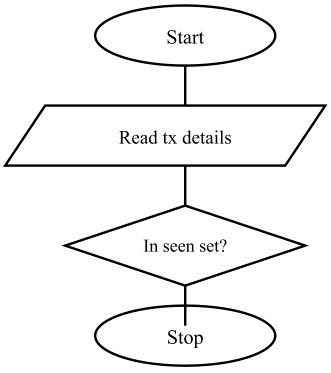
Algorithm :-

1. Start
2. Read n transactions
3. Maintain a set `seen` and list `duplicates`
4. For each transaction (account, amount, merchant): check if tuple is in `seen`
5. If present, append to `duplicates`; else add to `seen`
6. Print duplicates list
7. Stop

Pseudocode:

```
BEGIN
INPUT n
seen = empty set
duplicates = empty list
FOR i = 1 TO n DO
INPUT acc, amount, merchant
tx = (acc, amount, merchant)
IF tx IN seen THEN append tx to duplicates
ELSE add tx to seen
END FOR
PRINT duplicates
END
```

Flowchart:



Python Code (case6.py):

```
n = int(input("Enter number of transactions: "))
seen = set()
duplicates = []

for i in range(n):
    acc = input("Account ID: ")
    amt = float(input("Amount: "))
    merchant = input("Merchant: ")

    tx = (acc, amt, merchant)
    if tx in seen:
        duplicates.append(tx)
    else:
        seen.add(tx)

print("Duplicates Flagged:", len(duplicates))
print("Duplicate List:", duplicates)
```

Output:

```
Enter number of transactions: 3
Account ID: ACC101
Amount: 2500
Merchant: Amazon
Account ID: ACC102
Amount: 1000
Merchant: Flipkart
Account ID: ACC101
Amount: 2500
Merchant: Amazon
Duplicates Flagged: 1
Duplicate List: [('ACC101', 2500.0, 'Amazon')]
```

Conclusion: Flags repeated financial transactions to prevent duplicate processing errors.

---

7) Case Study 30: Student Email and Identifier Validation

Problem Statement: Validate student roll numbers and institutional email addresses using basic string operations and domain suffix checks.

Objectives: To verify character patterns and check string formatting using built-in string functions.

Assumptions: Valid roll numbers are 8 digits long and valid emails end with `@rvrnri.ac.in`.

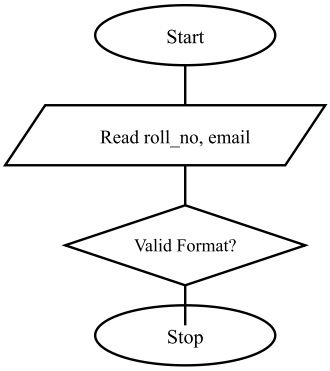
Algorithm :-

1. Start
2. Read roll_no and email
3. Check if roll_no is numeric and length == 8
4. Check if email ends with "@rvrnri.ac.in"
5. If both conditions pass, print "VALID", else print "INVALID"
6. Stop

Pseudocode:

```
BEGIN
INPUT roll_no, email
IF len(roll_no) == 8 AND roll_no IS DIGIT AND email ENDSWITH "@rvrnri.ac.in" THEN
    PRINT "VALID CREDENTIALS"
ELSE
    PRINT "INVALID CREDENTIALS"
END
```

Flowchart:



Python Code (case7.py):

```
roll_no = input("Enter Roll Number: ")
email = input("Enter Email Address: ").lower()

is_roll_valid = roll_no.isdigit() and len(roll_no) == 8
is_email_valid = email.endswith("@rvrnri.ac.in")

if is_roll_valid and is_email_valid:
    print("STATUS: VALID STUDENT CREDENTIALS")
else:
    print("STATUS: INVALID FORMAT DETECTED")
```

Output:

```
Enter Roll Number: 26000880
Enter Email Address: chinnam.nihanth@rvrnri.ac.in
STATUS: VALID STUDENT CREDENTIALS
```

Conclusion: Checks validity of student roll numbers and official email extensions.

---

8) Case Study 34: Student Feedback Sentiment Scoring

Problem Statement: Analyze course feedback textual comments to categorize student responses as Positive, Negative, or Neutral based on keyword scoring.

Objectives: To evaluate string content by splitting words and counting keyword matches.

Assumptions: Positive words = ["excellent", "good", "great", "helpful"], Negative = ["poor", "bad", "confusing"].

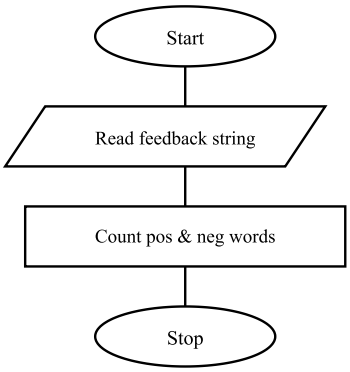
Algorithm :-

1. Start
2. Read feedback text string
3. Convert string to lower case and split into list of words
4. Count occurrences of positive words and negative words
5. If pos_count > neg_count print POSITIVE, else if neg_count > pos_count print NEGATIVE, else NEUTRAL
6. Stop

Pseudocode:

```
BEGIN
INPUT text
words = split(lowercase(text))
pos_score = count_matches(words, pos_words)
neg_score = count_matches(words, neg_words)
IF pos_score > neg_score THEN PRINT "POSITIVE"
ELSE IF neg_score > pos_score THEN PRINT "NEGATIVE"
ELSE PRINT "NEUTRAL"
END
```

Flowchart:



Python Code (case8.py):

```
feedback = input("Enter student feedback: ").lower()
words = feedback.split()

pos_words = ["excellent", "good", "great", "helpful", "clear"]
neg_words = ["poor", "bad", "confusing", "difficult", "slow"]

pos_score = sum(1 for w in words if w in pos_words)
neg_score = sum(1 for w in words if w in neg_words)

if pos_score > neg_score:
    sentiment = "POSITIVE"
elif neg_score > pos_score:
    sentiment = "NEGATIVE"
else:
    sentiment = "NEUTRAL"

print("Sentiment Result:", sentiment)
print("Score Delta:", pos_score - neg_score)
```

Output:

```
Enter student feedback: The course content was excellent and helpful
Sentiment Result: POSITIVE
Score Delta: 2
```

Conclusion: Analyzes student comments and classifies overall sentiment rating.

---

9) Case Study 44: Hospital Ward Occupancy Monitoring

Problem Statement: Monitor bed occupancy rates across hospital wards and raise capacity alerts when occupancy exceeds 85%.

Objectives: To compute percentages and evaluate capacity thresholds across hospital wards.

Assumptions: Total beds and occupied beds are non-negative integers.

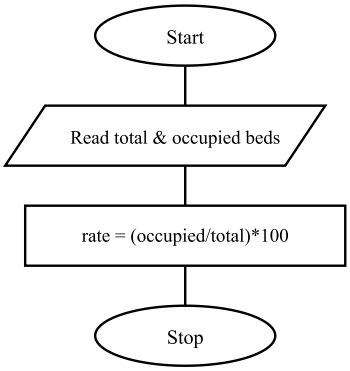
Algorithm :-

1. Start
2. Read number of wards w
3. For each ward: read ward_name, total_beds, occupied_beds
4. Calculate rate = (occupied_beds / total_beds) * 100
5. If rate >= 85.0 print "CRITICAL OCCUPANCY ALERT"
6. Print ward occupancy rate
7. Stop

Pseudocode:

```
BEGIN
INPUT w
FOR i = 1 TO w DO
INPUT ward_name, total_beds, occupied_beds
rate = (occupied_beds / total_beds) * 100
IF rate >= 85.0 THEN PRINT ward_name, "HIGH CAPACITY ALERT"
PRINT ward_name, rate
END FOR
END
```

Flowchart:



Python Code (case9.py):

```
num_wards = int(input("Enter number of wards: "))

for i in range(num_wards):
    ward_name = input("Ward Name: ")
    total_beds = int(input("Total Beds: "))
    occupied_beds = int(input("Occupied Beds: "))

    rate = (occupied_beds / total_beds) * 100
    print(ward_name, "Occupancy:", round(rate, 1), "%")

    if rate >= 85.0:
        print("ALERT: Ward", ward_name, "is at critical capacity!")
```

Output:

```
Enter number of wards: 2
Ward Name: ICU
Total Beds: 20
Occupied Beds: 18
ICU Occupancy: 90.0 %
ALERT: Ward ICU is at critical capacity!
Ward Name: General
Total Beds: 50
Occupied Beds: 35
General Occupancy: 70.0 %
```

Conclusion: Monitors hospital bed occupancy and alerts when wards exceed 85% capacity.

---

10) Case Study 58: Library Borrowing and Fine Reconciliation

Problem Statement: Calculate library overdue fines based on overdue days and member status (Student vs Faculty).

Objectives: To evaluate tiered fine structures using nested conditional logic.

Assumptions: Student base fine is Rs. 2/day, Faculty is Rs. 5/day. Fines increase by 50% after 7 days overdue.

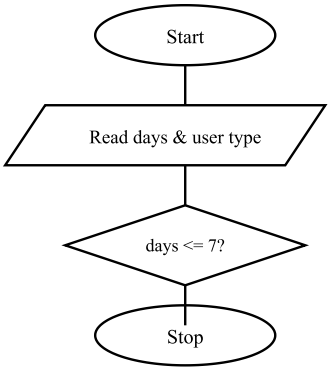
Algorithm :-

1. Start
2. Read days_overdue and is_student (YES/NO)
3. If days_overdue <= 0 set fine = 0
4. Set rate = 2.0 if is_student == "YES" else 5.0
5. If days_overdue <= 7 fine = days_overdue * rate
6. Else fine = (7 * rate) + ((days_overdue - 7) * rate * 1.5)
7. Print fine
8. Stop

Pseudocode:

```
BEGIN
INPUT days_overdue, user_type
rate = 2.0 IF user_type == "STUDENT" ELSE 5.0
IF days_overdue <= 7 THEN fine = days_overdue * rate
ELSE fine = (7 * rate) + ((days_overdue - 7) * rate * 1.5)
PRINT fine
END
```

Flowchart:



Python Code (case10.py):

```
days_overdue = int(input("Enter days overdue: "))
user_type = input("Enter user type (STUDENT/FACULTY): ").upper()

if days_overdue <= 0:
    fine = 0.0
else:
    rate = 2.0 if user_type == "STUDENT" else 5.0
    if days_overdue <= 7:
        fine = days_overdue * rate
    else:
        fine = (7 * rate) + ((days_overdue - 7) * rate * 1.5)

print("Total Overdue Fine: Rs.", round(fine, 2))
```

Output:

```
Enter days overdue: 10
Enter user type (STUDENT/FACULTY): STUDENT
Total Overdue Fine: Rs. 23.0
```

Conclusion: Calculates library overdue fines based on days overdue and member status.